

Personal Portfolio Website – AWS Serverless Architecture Documentation

1. Project Overview

This project is a **modern, fully responsive personal portfolio website** built to showcase professional skills, projects, experience, and certifications. It leverages **AWS serverless services** to implement:

- **Dynamic “Message Me” form** using **AWS Lambda + SES**
- **Cost-optimized temporary website hosting** using **AWS Lambda + EventBridge Scheduler**, allowing the website to activate for **30 minutes per session**
- **SNS notifications** to alert the portfolio owner whenever the website is started

This architecture demonstrates **scalable, cost-efficient cloud design** while providing a professional portfolio experience.

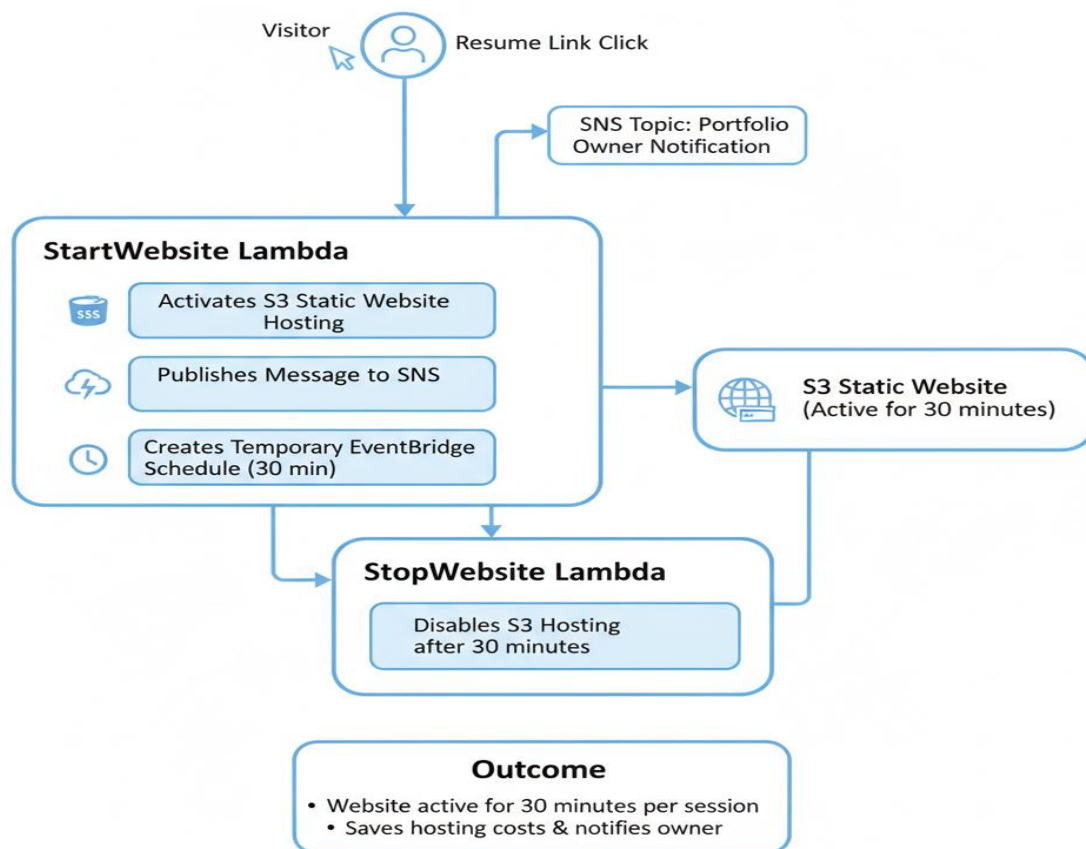
2. Objectives

- Showcase professional portfolio with a modern, interactive UI
 - Implement serverless backend functionality:
 - Contact form submission
 - Temporary website activation
 - Reduce hosting costs by limiting S3 website uptime
 - Notify the portfolio owner each time the website is accessed
 - Maintain a secure and professional environment for visitors
-

3. Architecture & Workflow

3.1 Website Activation Flow

1. **Visitor clicks the resume link**
2. **StartWebsite Lambda:**
 - Activates the S3 static website hosting
 - Publishes a message to **SNS**, notifying the portfolio owner
 - Creates a temporary **EventBridge schedule** to stop the site in 30 minutes
3. **StopWebsite Lambda:** Disables the S3 hosting after 30 minutes
4. **Outcome:** Website is only active for 30 minutes per session, saving hosting costs and notifying the owner

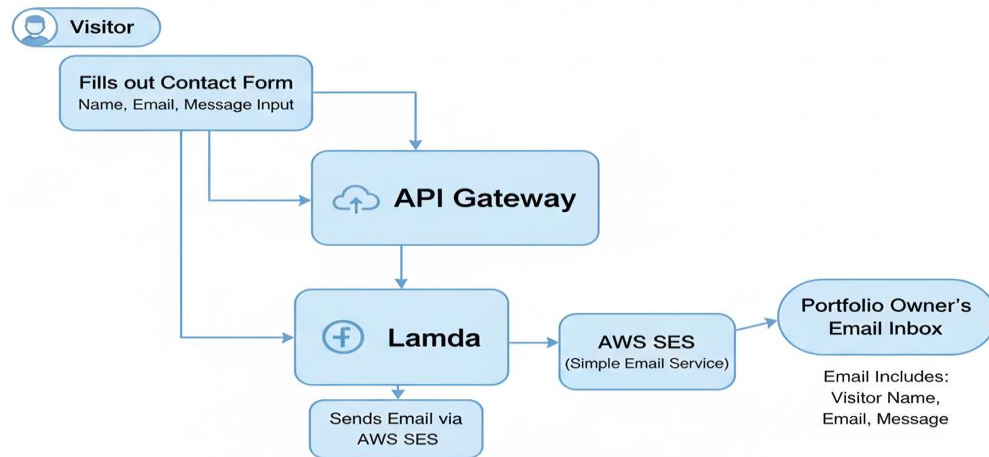


3.2 Contact Form Flow

1. Visitor fills out the **contact form**
2. **API Gateway** receives the request and triggers **Lambda**
3. **Lambda** sends an email via **AWS SES** to the portfolio owner

4. Emails include the visitor's input (name, email, message)

Contact Form Flow



4. AWS Services & Roles

Service	Purpose	Key Configurations / Notes
S3	Hosts static portfolio files	Public bucket with website hosting enabled
Lambda	Handles contact form & website activation	Two main functions: startWebsite & stopWebsite
EventBridge Scheduler	Automates website stop after 30 minutes	Schedules stopWebsite Lambda dynamically
API Gateway	Exposes HTTP endpoint for contact form	Handles CORS, routes requests to Lambda
SES	Sends emails from contact form	Verified email/domain required
SNS	Sends notification emails to owner	Triggered by startWebsite Lambda

5. Security & Cost Optimization

- **IAM Roles:** Restricted permissions for Lambda functions (least privilege)
 - **EventBridge Scheduler:** Automates stop to prevent unnecessary S3 costs
 - **S3 Free Tier:** 5 GB storage + 20K GET requests/month
 - **SNS Free Tier:** 1 million publishes/month + 1,000 email notifications/month
 - **SES:** Verified domain ensures secure and reliable email delivery
 - **Temporary Hosting:** Website only active when accessed via resume link
-

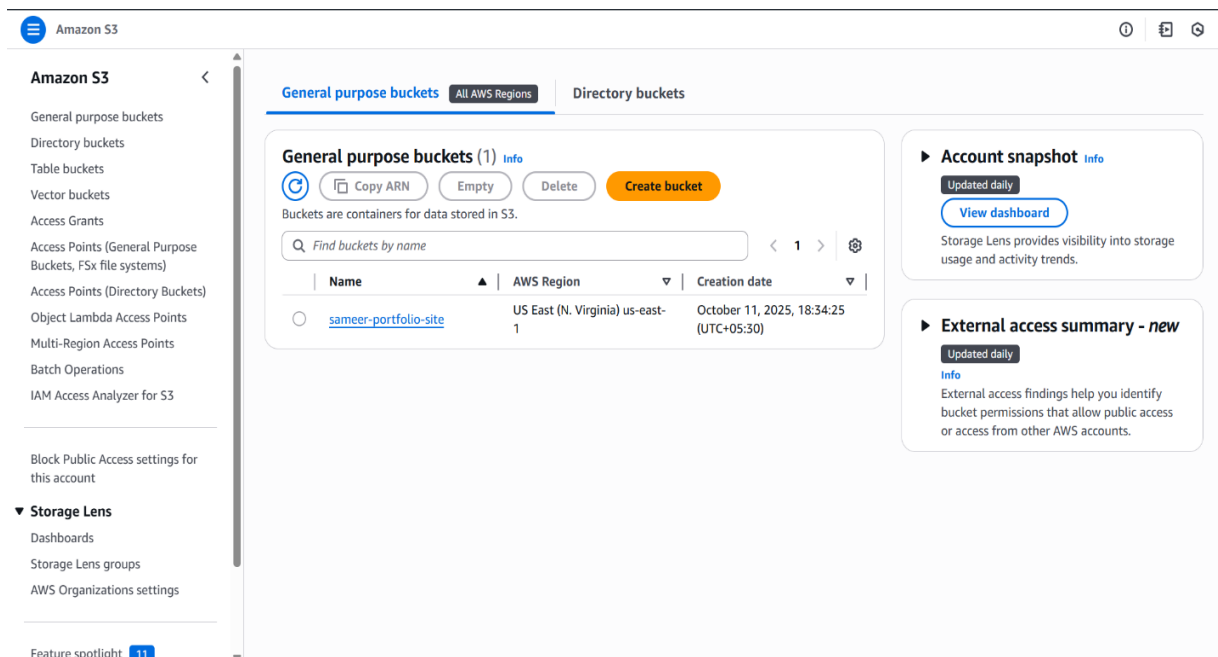
6. Features

- Temporary Portfolio Website Hosting (30 min/session)
 - Automatic activation and deactivation of S3 hosting
 - Dynamic Contact Form integrated with SES
 - SNS notifications to portfolio owner on site access
 - Modern UI/UX: Dark/Light mode, typing animation, scroll-based progress, interactive cards
 - Modular, serverless architecture for easy maintenance
-

7. Implementation Steps (GUI-Based)

7.1 S3 Setup

1. AWS Console → S3 → Create bucket → Name it uniquely (e.g., sameerm3-portfolio)
2. Enable **static website hosting** → Index document: index.html
3. Upload all portfolio files (HTML, CSS, JS, images)
4. Set bucket policy → Allow public read access



Static website hosting

Use this bucket to host a website or redirect requests. [Learn more](#)

Edit

We recommend using AWS Amplify Hosting for static website hosting
Deploy a fast, secure, and reliable website quickly with AWS Amplify Hosting. Learn more about [Amplify Hosting](#) or [View your existing Amplify apps](#)

Create Amplify app

S3 static website hosting

Enabled

Hosting type

Bucket hosting

Bucket website endpoint

When you configure your bucket as a static website, the website is available at the AWS Region-specific website endpoint of the bucket. [Learn more](#)

<http://sameer-portfolio-site.s3-website-us-east-1.amazonaws.com>

Amazon S3 > Buckets > sameer-portfolio-site

sameer-portfolio-site Info

Objects (5)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

☐	Name	Type	Last modified	Size	Storage class
☐	HireFusion-AI.png	png	October 15, 2025, 20:47:29 (UTC+05:30)	1.1 MB	Standard
☐	OnDemand-Portfolio.png	png	October 15, 2025, 20:47:32 (UTC+05:30)	979.0 KB	Standard
☐	portfolio.html	html	October 16, 2025, 14:55:07 (UTC+05:30)	46.6 KB	Standard
☐	sameer_resume.pdf	pdf	October 16, 2025, 14:55:08 (UTC+05:30)	175.7 KB	Standard
☐	sameer.png	png	October 15, 2025, 20:47:37 (UTC+05:30)	1017.2 KB	Standard

7.2 Create IAM Roles

Lambda Execution Role (SiteControlLambdaRole)

- Trusted entity: **Lambda**
- Attach policies: **AWSLambdaBasicExecutionRole**, **AmazonSNSFullAccess**

Identity and Access Management (IAM) > Roles > SiteControlLambdaRole

SiteControlLambdaRole Info

Allows Lambda functions to call AWS services on your behalf.

Summary

Creation date: October 11, 2025, 21:34 (UTC+05:30)

Last activity: 14 hours ago

ARN: [arn:aws:iam::785269092008:role/SiteControlLambdaRole](#)

Maximum session duration: 1 hour

Permissions policies (4) Info

You can attach up to 10 managed policies.

☐	Policy name	Type	Attached entities
☐	AllowPassRoleForScheduler	Customer inline	0
☐	AmazonSNSFullAccess	AWS managed	1
☐	AWSLambdaBasicExecutionRole	AWS managed	2
☐	SiteControlExtraPerms	Customer inline	0

Create inline policies: **AllowSchedulerRolePass** (

```
{
  "Version": "2012-10-17",
  "Statement": [
    {

```

```

        "Sid": "AllowSchedulerRolePass",
        "Effect": "Allow",
        "Action": [
            "iam:PassRole"
        ],
        "Resource": "arn:aws:iam::<Account_Id>:role/SchedulerInvokeLambdaRole"
    },
    {
        "Sid": "AllowSchedulerAndS3",
        "Effect": "Allow",
        "Action": [
            "scheduler:CreateSchedule",
            "scheduler:DeleteSchedule",
            "s3:PutBucketPolicy",
            "s3:DeleteBucketPolicy"
        ],
        "Resource": "*"
    }
]
}
),

```

SiteControlExtraPerms (

```

{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "s3:PutBucketPolicy",
                "s3:DeleteBucketPolicy"
            ],

```

```

        "Resource": "arn:aws:s3:::sameer-portfolio-site"
    },
    {
        "Effect": "Allow",
        "Action": [
            "scheduler:CreateSchedule",
            "scheduler>DeleteSchedule"
        ],
        "Resource": "*"
    }
]
} ).

```

EventBridge Scheduler Role (EventSchedulerInvokeRole)

- Trusted entity: **Custom trust policy** → Lambda can assume role

```

{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": {
                "Service": "scheduler.amazonaws.com"
            },
            "Action": "sts:AssumeRole"
        }
    ]
}

```

- Create inline policy: **invokelambda** (

```

{
    "Version": "2012-10-17",
    "Statement": [

```



```
{
  "Effect": "Allow",
  "Action": "lambda:InvokeFunction",
  "Resource": "*"
}
]
}
).
```

SES Role (MessageLambdaRole)

MessageLambdaRole [Info](#)

[Delete](#)

Allows Lambda functions to call AWS services on your behalf.

Summary [Edit](#)

Creation date

October 11, 2025, 22:02 (UTC+05:30)

Last activity

 2 days ago

ARN

 arn:aws:iam::785269092008:role/MessageLambdaRole

Maximum session duration

1 hour

Permissions

[Trust relationships](#)[Tags](#)[Last Accessed](#)[Revoke sessions](#)

Permissions policies (2) [Info](#)

You can attach up to 10 managed policies.

[Simulate](#)[Remove](#)[Add permissions](#)

Filter by Type

All types

< 1 > 

☐	Policy name 	Type	Attached entities
☐	 AWSLambdaBasicExecutionRole	AWS managed	2
☐	 SESAAllowSend	Customer inline	0

- Trusted entity: **Lambda**
- Attach policies: `AWSLambdaBasicExecutionRole`
- Create inline policy: `SESAAllowSend` (

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "ses:SendEmail",
      "Resource": "*"
    }
  ]
}
```

```
    }  
  ]  
}  
).
```

7.3 Lambda Functions

startWebsite Lambda

1. Create Lambda → Name: startWebsite → Runtime: Python 3.10
2. Role: SiteControlLambdaRole
3. Paste code to:
 - Enable S3 website
 - Schedule StopWebsite Lambda via EventBridge
 - Publish SNS notification
 - Code (

```
import boto3, json, os, uuid  
from datetime import datetime, timedelta, timezone  
  
sns = boto3.client('sns')  
s3 = boto3.client('s3')  
scheduler = boto3.client('scheduler')  
  
BUCKET = os.environ['BUCKET']  
SITE_URL = os.environ['SITE_URL']  
STOP_LAMBDA_ARN = os.environ['STOP_LAMBDA_ARN']  
SCHEDULER_ROLE_ARN = os.environ['SCHEDULER_ROLE_ARN']  
  
def lambda_handler(event, context):  
    # make site public  
    policy = {  
        "Version": "2012-10-17",  
        "Statement": [{  
            "Sid": "PublicReadGetObject",  
            "Effect": "Allow",  
            "Principal": "*",  
            "Action": ["s3:GetObject"],  
            "Resource": [f"arn:aws:s3:::{BUCKET}/*"]  
        }]  
    }
```

```

    }
    s3.put_bucket_policy(Bucket=BUCKET, Policy=json.dumps(policy))

    # schedule stop in 30 min
    run_time = datetime.now(timezone.utc) + timedelta(minutes=30)
    schedule_name = f"stop-website-{uuid.uuid4().hex[:8]}"
    scheduler.create_schedule(
        Name=schedule_name,
        ScheduleExpression=f"at({run_time.strftime('%Y-%m%dT%H:%M:%S')})",
        FlexibleTimeWindow={'Mode': 'OFF'},
        Target={
            'Arn': STOP_LAMBDA_ARN,
            'RoleArn': SCHEDULER_ROLE_ARN
        }
    )

    sns.publish(
        TopicArn='arn:aws:sns:us-east-1:785269092008:PortfolioSiteStarted',
        Message='Someone just started your portfolio website!',
        Subject='Portfolio Notification'
    )

    # redirect
    return {
        "statusCode":302,
        "headers":{"Location":SITE_URL},
        "body":json.dumps({"message":"Redirecting"})
    }
).

```

4. Add environment variables:

- BUCKET → your S3 bucket name
- REGION → your AWS region
- STOP_LAMBDA_ARN → stopWebsite Lambda ARN
- SCHEDULER_ROLE_ARN → EventSchedulerInvokeRole ARN

Environment variables (4)

[Edit](#)

The environment variables below are encrypted at rest with the default Lambda service key.

Key	Value
BUCKET	sameer-portfolio-site
SCHEDULER_ROLE_ARN	arn:aws:iam::785269092008:role/SchedulerInvokeLambdaRole
SITE_URL	http://sameer-portfolio-site.s3-website-us-east-1.amazonaws.com
STOP_LAMBDA_ARN	arn:aws:lambda:us-east-1:785269092008:function:stopWebsite

stopWebsite Lambda

- Creates Lambda → Name: stopWebsite → Runtime: Python 3.10
- Role: PortfolioLambdaExecRole
- Paste code to disable S3 website hosting
- Code (

```
import boto3, os, json
s3 = boto3.client('s3')
BUCKET = os.environ.get('BUCKET')

def lambda_handler(event, context):
    try:
        s3.delete_bucket_policy(Bucket=BUCKET)
    except:
        pass
    return {"statusCode":200,"body":json.dumps({"message":"Website
stopped"})}
).
```

5. Add environment variables:

- BUCKET → your S3 bucket name

Environment variables (1)

The environment variables below are encrypted at rest with the default Lambda service key.

 Find environment variables

Key	Value
BUCKET	sameer-portfolio-site

SESMessages Lambda

- Creates Lambda → Name: SendMessageSES → Runtime: Python 3.10
- Role: MessageLambdaRole
- Paste code to disable S3 website hosting
- Code (

```
import boto3
import json
import os
from botocore.exceptions import ClientError

# SES client (update region if needed)
ses = boto3.client('ses', region_name='us-east-1')

# Environment variables
FROM = os.environ['FROM'] # Must be verified in SES
TO = os.environ['TO']      # Your email, any recipient in production
mode

def lambda_handler(event, context):
    try:
        # Parse the incoming POST body
        body = json.loads(event.get('body', '{}'))
        name = body.get('name', 'Anonymous')
        email = body.get('email', 'No Email')
        message = body.get('message', 'No Message')

        # Compose email content
        subject = f"Portfolio Message from {name}"
        email_body = f"Name: {name}\nEmail: {email}\n\nMessage:\n{message}"

        # Send email via SES
        response = ses.send_email(
            Source=FROM,
            Destination={'ToAddresses': [TO]},
            Message={
                'Subject': {'Data': subject},
                'Body': {'Text': {'Data': email_body}}
            }
        )

        # Success response
        return {
            'statusCode': 200,
```

```

        'body': json.dumps({'message': 'Message sent
successfully!'})
    }

    except ClientError as e:
        # SES error
        print(f"SES Error: {e.response['Error']['Message']}")
        return {
            'statusCode': 500,
            'body': json.dumps({
                'error': 'Failed to send message via SES',
                'details': e.response['Error']['Message']
            })
        }

    except Exception as e:
        # General error
        print(f"Error: {str(e)}")
        return {
            'statusCode': 500,
            'body': json.dumps({'error': 'Internal server error',
'details': str(e)})
        }
    ).

```

6. Add environment variables:

- From → Mail verified in SES
- To → Recipient mail

Environment variables (2)

The environment variables below are encrypted at rest with the default Lambda service key.

 Find environment variables

Key	Value
FROM	sayyadsameerM3@gmail.com
TO	sayyadsameerM3@gmail.com

7.4 API Gateway

1. Create **HTTP API** → Add integrations → startWebsite Lambda, sendMessageSES Lambda
2. Route: /start (method GET), /contact(method POST)
3. Enable **CORS**
4. Deploy → Note Invoke URL

APIs (1/1)

◀ 1 ▶

⚙

Name	Description	ID	Protocol	API endpoint type	Created
websiteapi		2dubub6pbj	HTTP	Regional	2025-10-11

Routes for websiteapi

Create

▼ /contact

POST

▼ /start

GET

Route details

GET /start (ID: bw66myg)

ARN

[arn:aws:apigateway:us-east-1::apis/2dubub6pbj/routes/bw66myg](#)

Authorization

Authorizers protect your API against unauthorized requests. Routes with no authorization attached are open.

No authorizer attached to this route.

Attach authorization

Integration

The integration is the backend resource that this route calls when it receives a request.

7ijk4a3

Configure

Delete

Edit

Integrations

Stage: - ▼

Deploy

Attach integrations to routes

Manage integrations

Integrations for websiteapi

Create

☒ sendMessageSES (c8034gk)

☐ startWebsite (7ijk4a3)

Integration details

sendMessageSES

Lambda function

sendMessageSES (us-east-1)

Integration ID

c8034gk

Description

-

Delete

Edit

Configure CORS

Info

Configure

Clear

CORS allows resources from different domains to be loaded by browsers. If you configure CORS for an API, API Gateway ignores CORS headers returned from your backend integration. See our [CORS documentation](#) for more details.

Access-Control-Allow-Origin

* ✕

Access-Control-Allow-Headers

content-type ✕

Access-Control-Allow-Methods

POST ✕ OPTIONS ✕

Access-Control-Expose-Headers

No Expose Headers are allowed

Access-Control-Max-Age

0 Seconds

Access-Control-Allow-Credentials

☐ NO

Stages

Stages for websiteapi Create

☒ \$default

Stage details Delete Edit

Details

Name	Created	Last updated
\$default	October 11, 2025 9:50 PM	October 11, 2025 10:27 PM
Invoke URL	https://2dubub6pbj.execute-api.us-east-1.amazonaws.com	

7.5 EventBridge Scheduler

Lambda creates a schedule for it to trigger another **Lambda** later

7.6 SNS Notifications

- Create SNS Topic → PortfolioAccessTopic
- Add **Email subscription** → confirm in inbox
- startWebsite Lambda publishes message to this topic whenever website starts

Topics (1) Edit Delete Publish message Create topic

Name	Type	ARN
PortfolioSiteStarted	Standard	arn:aws:sns:us-east-1:785269092008:Portfolio...

Subscriptions (1) Edit Delete Request confirmation Confirm subscription Create subscription

ID	Endpoint	Status	Protocol	Topic
Deleted	sayyadsameerM3@gmail...	Confirmed	EMAIL	PortfolioSiteStarted

7.7 SES Configuration

- Verify your email/domain
- Lambda uses **SES sendEmail** API for contact form submissions

Identities

The **Identities** pane lists your domains, subdomains, and email address identities. All identities must be verified before you use them to send email in Amazon SES. [Learn more](#). The **Recommendations** pane lists high-impact email authentication issues found for the identities you select and check for recommendations. [Learn more](#)

The screenshot shows the 'Identities (1)' pane in the AWS IAM console. It includes a search bar, a table with columns for Identity, Identity type, and Identity status, and buttons for 'Check for recommendations', 'Send test email', 'Delete', and 'Create identity'. The table lists one identity: 'sayyadsameerM3@gmail.com' with a status of 'Verified'.

Identity	Identity type	Identity status
sayyadsameerM3@gmail.com	Email address	Verified

7.8 Testing & Debugging

1. Click resume link → S3 website starts
2. Check SNS email → notification received
3. Wait 30 minutes → website stops automatically
4. Submit contact form → email delivered via SES
5. Debugging and Monitoring → via Cloud Watch

Log groups (3)

By default, we only load up to 10000 log groups.

The screenshot shows the 'Log groups (3)' pane in the AWS CloudWatch console. It includes a search bar, a table with columns for Log group, Log class, Anomaly d..., Data p..., Sensit..., Retenti..., and Metric, and buttons for 'Actions', 'View in Logs Insights', 'Start tailing', and 'Create log group'. The table lists three log groups: '/aws/lambda/sendMessageSES', '/aws/lambda/startWebsite', and '/aws/lambda/stopWebsite', all with a status of 'Standard'.

Log group	Log class	Anomaly d...	Data p...	Sensit...	Retenti...	Metric
/aws/lambda/sendMessageSES	Standard	Configure	-	-	Never exp...	-
/aws/lambda/startWebsite	Standard	Configure	-	-	Never exp...	-
/aws/lambda/stopWebsite	Standard	Configure	-	-	Never exp...	-

8. Advantages

- **Cost-efficient:** Runs only when needed
- **Serverless & Scalable:** Lambda and EventBridge manage backend without servers
- **Recruiter-friendly:** Anyone can access and submit forms without AWS account
- **Secure & Reliable:** SES verified domain, SNS notifications, IAM roles
- **Maintainable:** Modular architecture allows easy updates or expansion